

Unity ECS 기반
대규모 3D 인스턴스 렌더링 성능 분석 및 최적화



팀명: 미오

202255658 김재식 rla0578@pusan.ac.kr

202155593 이천서 angel5337@pusan.ac.kr

202255568 안지홍 anjhu@pusan.ac.kr

지도교수: 박영진

제출일: 2026년 5월 14일

부산대학교

목차

1 프로젝트 개요	1
1.1 프로젝트 소개	1
1.2 개발 배경 및 필요성	1
1.3 프로젝트 목표	2
1.4 프로젝트 적합성	3
2 요구조건 및 현실적 제약 분석	4
2.1 대상 문제 정의	4
2.2 현실적 제약 사항과 대책	6
2.3 성공 기준	7
3 관련 기술 분석	8
3.1 데이터 지향 설계와 Unity ECS	8
3.2 Burst Compiler와 Job System	8
3.3 BRG와 GPU Indirect Rendering	9
3.4 Compute Shader와 GPU 기반 데이터 처리	9
3.5 기술 선택 이유	10
4 시스템 설계	11
4.1 전체 시스템 아키텍처	11
4.2 ECS 데이터 흐름	11
4.3 군집형 인스턴스 시뮬레이션 설계	12
4.4 렌더링 파이프라인 설계	13
4.5 실험 조건 구성	14
5 구현 계획	15
5.1 현재 프로젝트 기반	15
5.2 구현 단계	16

5.3	설계상 핵심 결정	18
5.4	기능 명세	19
6	실험 및 평가 계획	20
6.1	실험 범위 설정	20
6.2	실험 변인	20
6.3	측정 지표	21
6.4	실험 조건	22
6.5	실험 절차	23
6.6	기대 결과와 해석 방법	24
6.7	결과 기록 방식	24
6.8	보고서 반영 방식	25
7	추진 체계 및 역할 분담	26
7.1	추진 일정	26
7.2	역할 분담	28
7.3	협업 방식	29
8	기대 효과 및 향후 발전 방향	30
8.1	기대 효과	30
8.2	향후 발전 방향	30
	참고 문헌 및 자료	32

제1장 프로젝트 개요

1.1 프로젝트 소개

본 프로젝트는 Unity 6 환경에서 대규모 3D 인스턴스 장면을 실시간으로 처리하기 위한 고성능 렌더링 제출 구조와 시뮬레이션 데이터 파이프라인을 설계하고 검증하는 프로젝트이다. 기존 GameObject와 MonoBehaviour 중심의 구현은 오브젝트 수가 늘어날수록 프레임당 Update() 호출, 객체 접근, 렌더 제출 비용이 함께 증가한다. 이로 인해 수천에서 수만 단위의 3D 인스턴스, 논리 유닛, 군집형 인스턴스가 동시에 등장하는 장면에서는 CPU 업데이트 병목과 Render Thread 병목이 빠르게 드러난다.

본 프로젝트는 이 문제를 단순히 그래픽 품질을 낮추어 회피하지 않고, 데이터 지향 설계와 GPU 기반 제출 파이프라인을 결합하여 분석하고자 한다. 핵심 구현 방향은 BatchRendererGroup(BRG) 및 GPU Indirect Rendering을 중심으로 한 대규모 인스턴스 렌더링 최적화이며, Unity ECS, Burst Compiler, Job System은 논리 유닛과 인스턴스 상태를 효율적으로 관리하기 위한 기술적 기반으로 활용한다. 최종 결과물은 대규모 3D 인스턴스 시뮬레이션 및 렌더링 프로토타입, 성능 측정 결과, 그리고 BRG 제출 방식에 따른 병목 이동을 분석하는 기술 보고서로 구성된다.

1.2 개발 배경 및 필요성

현대 실시간 그래픽스 응용 및 시뮬레이션 콘텐츠는 시각적으로 풍부한 장면과 동시에 안정적인 상호작용성을 요구한다. 특히 대규모 군집 표현, 공간 시뮬레이션, 전략적 시각화, 디지털 트윈과 같은 분야에서는 화면에 표시되는 개체 수가 증가하더라도 프레임타임이 예측 가능하게 유지되어야 한다. 그러나 일반적인 객체 지향 방식은 개별 객체마다 상태, 컴포넌트, 스크립트 호출이 분산되어 있어 캐시 효율이 낮고, 렌더링 제출 또한 객체 단위로 쪼개지기 쉽다.

이 프로젝트가 해결하려는 문제는 다음 세 가지로 요약된다.

- CPU 데이터 갱신 병목: 개별 오브젝트 또는 논리 유닛마다 상태 갱신을 수행하면 객체

수에 비례하여 Main Thread 부하가 증가한다.

- **렌더 제출 병목:** 많은 Mesh를 개별적으로 제출하면 Draw Call 또는 Draw Command 구성 비용이 증가하여 Render Thread가 병목이 될 수 있다.
- **측정 및 재현의 어려움:** 단순 FPS만 확인하면 병목이 CPU 데이터 갱신, CPU 렌더 제출, GPU 처리 중 어디에서 발생하는지 분해하기 어렵다.

개별 GameObject 또는 per-object draw 방식의 구조적 한계는 선행 벤치마크와 기존 렌더링 파이프라인 특성을 통해 확인할 수 있다. 따라서 본 프로젝트는 해당 방식을 직접 구현하는 데 범위를 소모하지 않고, BRG 기반 렌더링 제출 구조 내부에서 조건을 달리했을 때 병목이 어떻게 이동하는지를 분석하는 데 집중한다.

1.3 프로젝트 목표

본 프로젝트의 목표는 Unity 6 기반 대규모 3D 인스턴스 장면에서 BRG 및 GPU Indirect Rendering 중심의 렌더링 제출 구조를 구현하고, ECS 기반 논리 유닛 관리 구조와 결합하여 BRG 기반 실험 조건별 성능 차이를 정량적으로 분석하는 것이다. 특히 정적 데이터 기반 BRG(Render-only BRG), ECS 연계 BRG(ECS-fed BRG), 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건을 비교하여 Main Thread, Render Thread, GPU 단계의 병목 이동을 확인한다.

표 1.1: 프로젝트 핵심 목표

구분	목표	성공 기준
시뮬레이션 데이터 구조	논리 유닛 단위 ECS 데이터 모델과 병렬 데이터 갱신 구조 구현	Render-only BRG 조건과 ECS-fed BRG 조건을 비교, 논리 유닛 수 증가에 따른 Main Thread 비용을 분리하여 측정
렌더링 제출 구조	BRG 기반 Draw List 구성과 GPU Indirect Rendering 적용	Draw Command 구성 비용과 visible instance 관리 방식을 분석, visible instance 수 중심으로 렌더링 대상을 제어
실험 조건 구성	인스턴스 유형별 분할 제출과 통합 Draw List 제출 조건 구성	인스턴스 유형 수 증가에 따른 Render Thread, GPU Time, Draw Command Count 변화 비교
측정 체계	인스턴스 수, FPS, frame time, Draw Command, visible instance count, CPU/GPU 병목 지표 기록	동일 조건 반복 측정이 가능한 실험표와 로그 산출
결과물	실행 가능한 Unity 프로토타입과 성능 분석 보고서 작성	중간 및 최종보고서에서 구현 진척, 실험 결과, 병목 이동 분석 제시

1.4 프로젝트 적합성

본 과제는 교과목 과제 수준의 단일 기능 구현이 아니라, 실제 게임 엔진 환경에서 발생하는 대규모 3D 인스턴스 처리와 렌더링 제출 한계를 분석하고 시스템 구조를 설계하는 종합 설계과제이다. 자료구조, 컴퓨터 구조, 운영체제의 병렬 처리, 그래픽스 파이프라인, 소프트웨어 공학의 실험 설계 지식을 함께 적용한다. 또한 구현 결과를 정량 지표로 검증하므로 단순 데모가 아니라 공학적 판단 근거를 가진 시스템 구현 과제로 볼 수 있다.

제2장 요구조건 및 현실적 제약 분석

2.1 대상 문제 정의

대상 문제는 “대규모 3D 인스턴스 장면을 Unity에서 실시간으로 안정적으로 표현하고, BRG 기반 렌더링 제출 구조의 병목을 정량적으로 분석할 수 있는가”이다. 여기서 실시간성은 평균 FPS만을 의미하지 않는다. 사용자가 카메라를 이동하거나 논리 유닛의 상태가 갱신될 때도 프레임타임이 급격하게 튀지 않아야 하며, 렌더링 인스턴스 수와 논리 유닛 수가 증가할 때 병목 위치를 CPU 데이터 갱신, Render Thread 제출, GPU 처리 단계로 분해할 수 있어야 한다.

개별 GameObject 또는 per-object draw 방식은 인스턴스 수 증가에 따라 draw call과 CPU 제출 비용이 선형적으로 증가하는 구조적 한계를 가진다. 해당 한계는 교수자 제공 OpenGL 예비 벤치마크와 기존 렌더링 파이프라인 특성을 통해 선행 검토 대상으로 활용하고, 본 프로젝트의 직접 구현 범위에는 포함하지 않는다. 따라서 본 프로젝트는 기존 방식 재현보다 BRG 기반 렌더링 제출 구조 내부에서 Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건을 비교하는 데 집중한다.

본 프로젝트는 아래와 같이 문제를 세부 요구조건으로 나눈다.

표 2.1: 기능 및 품질 요구조건

분류	요구조건	설명	우선순위
시물레이션 데이터	군집형 인스턴스 상태 관리	개별 인스턴스를 모두 독립 GameObject로 두지 않고 논리 유닛 단위 엔티티와 내부 버퍼 데이터로 관리	높음

분류	요구조건	설명	우선순위
시뮬레이션 데이터	ECS 기반 데이터 갱신	이동, 회전, 배치 형태 전환, 내부 인스턴스 오프셋 계산을 ECS 기반 데이터 갱신 구조로 처리	높음
공간 처리	공간 질의 및 관심 영역 선정	공간 그리드 기반으로 주변 인스턴스, 관심 영역, 장애물 정보를 탐색하고 우선순위 또는 페널티 반영	중간
렌더링 제출	Render-only BRG 조건 구성	정적 또는 단순 인스턴스 데이터를 사용하여 순수 BRG 렌더링 제출 비용을 측정할 수 있는 조건 구성	높음
렌더링 제출	ECS-fed BRG 조건 구성	ECS 기반 데이터 갱신 결과를 BRG 제출 경로에 연결하여 논리 유닛 수 증가에 따른 Main Thread 부하 확인	높음
렌더링 제출	제출 방식 비교	인스턴스 유형별 분할 제출과 통합 Draw List 제출 조건을 구성하여 Draw Command 및 Render Thread 비용 차이 비교	높음
측정	반복 가능한 벤치마크	렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, Shadow 조건, 제출 방식별 성능 로그를 동일 조건에서 수집	높음

분류	요구조건	설명	우선순위
사용성	관찰 가능한 데모	카메라 이동, 군집형 인스턴스 이동, 배치 형태 전환, 렌더링 결과가 시각적으로 확인되어야 함	중간

2.2 현실적 제약 사항과 대책

표 2.2: 현실적 제약 및 대응 방안

제약 사항	영향	대응 방안
Unity 6 및 ECS API 변화	Entities, Rendering API가 버전에 따라 예제가 달라 구현 리스크 있음.	프로젝트 버전을 Unity 6000.3.9f1로 고정하고 패키지 버전을 보고서에 명시
BRG 구현 난이도	BRG는 일반 Renderer보다 Draw Command, Batch, Buffer 관리가 직접적이므로 초기 구현 비용이 높음.	Render-only BRG 조건을 먼저 구성한 뒤 ECS-fed BRG와 제출 방식 비교 조건으로 확장
개발 장비 성능 차이	측정값이 장비에 따라 달라 절대 성능 비교가 어려움.	동일 장비 내 상대 비교를 기본으로 하고 CPU, GPU, 해상도, Shadow 설정 조건 기록
GPU 병목 분리 어려움	FPS만으로는 렌더 제출 병목과 GPU 처리 병목을 구분하기 어려움.	Unity Profiler, Frame Debugger, Draw Command 수, visible instance 수, GPU time 함께 기록
대조군 구현 범위 문제	per-object draw 또는 MonoBehaviour 기반 대조군을 직접 구현하면 개발 및 연구 범위가 분산될 수 있음	해당 방식은 선행 벤치마크와 기술적 한계 분석으로 처리, 직접 구현은 BRG 기반 실험 조건에 집중

2.3 성공 기준

최종 성공 기준은 다음과 같다.

1. ECS 기반 논리 유닛 및 군집형 인스턴스 데이터 갱신 구조가 Unity 씬에서 정상 동작한다.
2. Render-only BRG 조건을 구성하여 시뮬레이션 부하가 최소화된 상태의 순수 렌더링 제출 비용을 측정할 수 있다.
3. ECS-fed BRG 조건을 구성하여 논리 유닛 수 증가가 Main Thread 및 BRG 제출 구조에 미치는 영향을 측정할 수 있다.
4. 인스턴스 유형별 분할 제출과 통합 Draw List 제출 조건을 구성하여 Draw Command, Render Thread, GPU Time의 차이를 비교할 수 있다.
5. Shadow ON을 기본 조건으로 측정하고, 대표 조건에서 Shadow OFF 보조 측정을 수행하여 ShadowCaster Pass의 영향을 분리할 수 있다.
6. 렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, 제출 방식 변화에 따른 FPS, frame time, Draw Command, visible instance count, CPU/GPU 병목 지표를 표와 그래프로 정리한다.

제3장 관련 기술 분석

3.1 데이터 지향 설계와 Unity ECS

데이터 지향 설계는 객체의 행동보다 데이터 배치와 접근 패턴에 초점을 둔다. Unity ECS는 Entity, Component, System을 분리하여 동일한 컴포넌트 구성을 가진 데이터를 청크 단위로 배치한다. 이 구조는 대량의 유사 데이터를 반복 처리할 때 캐시 효율을 높이고, Job System과 결합하여 멀티코어 병렬 처리를 구성할 수 있게 한다.

본 프로젝트에서 ECS는 BRG 기반 렌더링 실험에 필요한 논리 유닛 상태와 인스턴스 데이터를 안정적으로 공급하는 기술적 기반이다. 구체적으로는 논리 유닛 데이터 모델, 군집 파라미터, 상태 관리 정보, 이동 목표, 내부 인스턴스 슬롯, 로컬 위치 데이터를 데이터 중심 구조로 관리한다. 즉, ECS는 최종 연구의 중심 주제라기보다 대규모 3D 인스턴스의 위치, 방향, 상태 데이터를 일관되게 생성하고 갱신하기 위한 시뮬레이션 데이터 계층으로 활용된다.

특히 본 프로젝트에서는 ECS 기반 데이터 갱신 계층을 Render-only BRG 조건과 구분하여 분석한다. Render-only BRG 조건은 정적 또는 단순 인스턴스 데이터를 사용해 순수 렌더링 제출 비용을 측정하고, ECS-fed BRG 조건은 ECS 기반 데이터 갱신 결과를 BRG 제출 경로에 연결하여 논리 유닛 수 증가가 Main Thread에 미치는 영향을 확인하는 방식으로 구성한다.

3.2 Burst Compiler와 Job System

Burst Compiler는 C# Job 코드를 네이티브 수준으로 최적화하여 실행할 수 있게 한다. 본 프로젝트에서는 이동, 회전, 군집 내부 위치 보정, 공간 질의와 같은 반복 계산을 Burst Job으로 구성한다. 특히 IJobEntity, IJobChunk, NativeArray, NativeParallelMultiHashMap을 활용하면 많은 논리 유닛과 인스턴스 데이터를 병렬로 처리하면서도 메모리 할당과 동기화 비용을 통제할 수 있다.

본 기술은 렌더링 제출 구조 자체를 대체하기 위한 것이 아니라, BRG와 GPU Indirect Rendering에 전달할 인스턴스 데이터의 갱신 비용을 낮추기 위한 기반 기술이다. 따라서 실험에서는 렌더링 인스턴스 수 증가에 따른 Render Thread 및 GPU Time뿐 아니라, 논리

유닛 수 증가가 Main Thread에 미치는 영향도 함께 관찰한다.

3.3 BRG와 GPU Indirect Rendering

GPU Indirect Rendering은 CPU가 매 프레임 모든 인스턴스를 개별적으로 제출하지 않고, GPU 버퍼에 저장된 명령과 인스턴스 데이터를 기반으로 렌더링을 수행하는 방식이다. 개별 GameObject 또는 per-object draw 방식은 인스턴스 수가 증가할수록 draw call과 CPU 제출 비용이 선형적으로 증가하는 구조적 한계를 가진다. 이 한계는 교수자 제공 OpenGL 예비 벤치마크와 기존 렌더링 파이프라인 특성을 통해 선행 검토 대상으로 활용하고, 본 프로젝트의 직접 구현 범위에는 포함하지 않는다.

본 프로젝트의 핵심은 BRG 기반 제출 구조와 GPU Indirect Rendering 계열 최적화를 통해 대규모 3D 인스턴스의 Draw Command 증가를 억제하고, Main Pass와 ShadowCaster Pass에서 발생하는 제출 비용과 GPU 처리 비용을 정량적으로 분석하는 것이다. 이를 위해 본 프로젝트에서는 Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건을 구성한다.

여기서 인스턴스 유형은 모델, 머티리얼, 모션, 논리구조가 서로 다른 3D 인스턴스 그룹을 의미한다. 이 중 Mesh, Material, Shader, LOD 구성처럼 렌더링 제출 단위에 직접 영향을 주는 요소는 렌더 타입으로 구분할 수 있다. 본 착수보고서에서는 전반적인 실험 변인을 설명할 때는 인스턴스 유형을 기본 용어로 사용하고, 렌더링 제출 단위의 차이를 강조할 때만 렌더 타입을 사용한다.

3.4 Compute Shader와 GPU 기반 데이터 처리

Compute Shader는 렌더링 이전 단계에서 인스턴스 데이터 처리, 프리스텀 컬링, 행렬 생성, visible instance count 산출과 같은 작업을 GPU에서 수행할 수 있게 한다. 본 프로젝트에서는 ECS 기반 시뮬레이션 데이터 계층 또는 정적 인스턴스 데이터에서 생성된 위치와 방향 정보를 GPU 버퍼로 전달하고, GPU 기반 컬링 및 행렬 생성 과정을 통해 실제 렌더링 대상 인스턴스를 선별하는 구조를 계획한다.

이 구조는 CPU가 모든 인스턴스의 행렬을 직접 계산하고 제출하는 방식보다 대규모 인스턴스 처리에 적합하다. 또한 컬링 이후 visible instance count를 기록하면 실제 렌더링 대상 수와 Draw Command Count를 함께 분석할 수 있어, 병목이 데이터 갱신 단계, 렌더 제출 단계, GPU 처리 단계 중 어디에서 발생하는지 분리해 해석할 수 있다.

표 3.1: 핵심 기술과 프로젝트 적용 방식

기술	적용 위치	기대 효과
Unity ECS	논리 유닛, 군집 파라미터, 인스턴스 버퍼 관리	데이터 접근 패턴 단순화, 대량 인스턴스 상태 관리 비용 절감
Burst/Job System	이동, 배치 형태 전환, 공간 질의, 인스턴스 로컬 시뮬레이션	CPU 멀티코어 활용 및 Main Thread 부담 감소
Compute Shader	인스턴스 행렬 생성, 프러스트뮬 컬링, visible instance count 산출	CPU 측 렌더링 전처리 비용 감소 및 GPU 기반 데이터 처리
BRG/GPU Indirect Rendering	BRG Draw Command 및 Indirect Rendering 기반 대량 제출	Draw Command 증가 억제 및 대규모 3D 인스턴스 렌더링 가능성 확보
Profiler 기반 측정	실험별 frame time, Draw Command, visible instance count, GPU time 기록	병목 분해와 제출 방식별 개선 효과 검증

3.5 기술 선택 이유

본 프로젝트는 화면에 보이는 인스턴스 수가 많고, 개체의 상태 갱신 패턴이 유사하며, 매 프레임 반복 계산과 렌더링 제출이 함께 발생하는 유형의 문제이다. 따라서 개별 객체를 풍부하게 모델링하는 OOP 방식만으로는 대규모 장면에서 CPU 업데이트 비용과 Render Thread 제출 비용을 분리해 분석하기 어렵다.

ECS와 Burst/Job System은 논리 유닛과 인스턴스 데이터를 효율적으로 갱신하기 위한 기반으로 적합하며, BRG와 GPU Indirect Rendering은 대규모 3D 인스턴스의 제출 비용을 통제하고 병목 이동을 분석하기 위한 핵심 기술로 적합하다. 특히 본 프로젝트는 단순히 기존 방식보다 빠른 결과를 제시하는 것이 아니라, Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건을 비교하여 병목이 Main Thread, Render Thread, GPU 단계 중 어디로 이동하는지를 분석하는 데 중점을 둔다.

제4장 시스템 설계

4.1 전체 시스템 아키텍처

전체 구조는 입력 및 명령 계층, ECS 기반 시뮬레이션 데이터 계층, 공간 처리 계층, BRG 기반 렌더링 제출 계층, 측정 계층으로 나뉜다. 입력 계층은 카메라 이동과 논리 유닛 제어 입력을 단순한 명령 데이터로 변환하고, ECS 기반 시뮬레이션 데이터 계층은 이를 틱 단위로 적용한다. 렌더링 제출 계층은 시뮬레이션 결과 또는 정적 인스턴스 데이터를 GPU 버퍼로 전달하고, BRG 및 GPU Indirect Rendering 기반 구조를 통해 대규모 3D 인스턴스를 효율적으로 제출한다.

본 프로젝트는 개별 GameObject 또는 per-object draw 방식의 한계를 직접 구현하여 재현하는 것보다, BRG 기반 제출 구조 내부에서 어떤 조건이 병목을 유발하는지 분석하는데 초점을 둔다. 따라서 시스템 설계는 Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건을 동일한 실험 환경에서 비교할 수 있도록 구성한다.

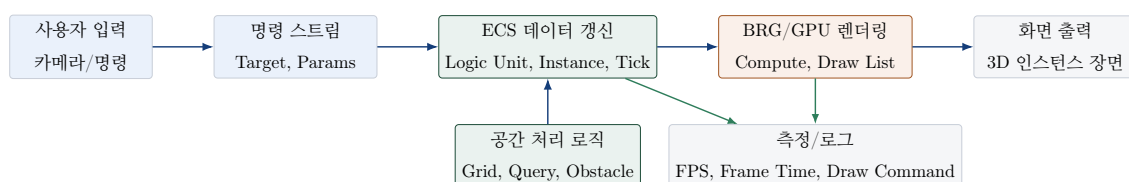


그림 4.1: 전체 시스템 아키텍처

4.2 ECS 데이터 흐름

ECS 데이터 모델은 논리 유닛 단위 엔티티를 중심으로 한다. 논리 유닛 엔티티는 위치, 속도, 상태, 이동 목표, 군집 파라미터를 컴포넌트로 가지고, 내부 렌더링 인스턴스의 상대 위치와 상태는 버퍼 형태로 저장된다. 이 방식은 모든 렌더링 인스턴스를 독립 GameObject로 분해하는 방식보다 관리 단위가 명확하고, 논리 유닛 내부의 상대 위치 계산을 버퍼 순회로 처리할 수 있다. 또한 BRG 기반 렌더링 계층에 전달할 위치, 방향, 가시성 관련 데이터를 일관된 구조로 생성할 수 있다.

ECS 데이터 흐름은 BRG 렌더링 실험에서 두 가지 역할을 가진다. 첫째, Render-only BRG 조건과 비교하기 위한 시뮬레이션 데이터 갱신 부하를 제공한다. 둘째, ECS-fed BRG 조건에서 논리 유닛 수(N_{unit}) 증가가 Main Thread와 렌더링 제출 구조에 미치는 영향을 관찰할 수 있게 한다.

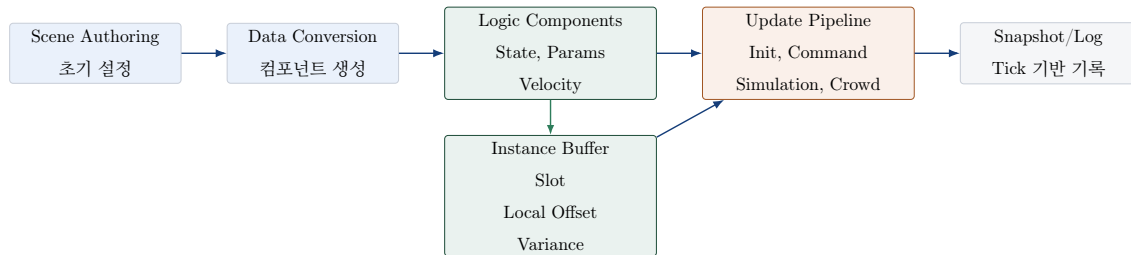


그림 4.2: ECS 데이터 흐름도

4.3 군집형 인스턴스 시뮬레이션 설계

군집형 인스턴스 시뮬레이션은 논리 유닛의 거시적 상태 갱신과 내부 인스턴스의 미시적 위치 보정을 분리한다. 논리 유닛 갱신 계층은 목표 방향, 회전, 속도, 배치 형태 전환 상태를 계산한다. 내부 인스턴스 갱신 계층은 각 인스턴스의 로컬 위치, 간격 유지, 분리 힘, 이동 지연, 방향을 계산한다. 이를 통해 논리 유닛 전체는 하나의 상태와 명령을 따르면서도 내부 인스턴스는 시각적으로 자연스러운 군집 움직임을 보인다.

이 계층은 연구의 중심인 BRG 렌더링 실험에 필요한 인스턴스 위치 및 방향 데이터를 생성하는 기반으로 활용된다. 다만 실험 해석에서는 군집형 인스턴스 시뮬레이션 자체의 완성도보다, 이 데이터 갱신 계층이 BRG 제출 구조와 결합되었을 때 Main Thread와 Render Thread에 어떤 부하를 추가하는지에 초점을 둔다.

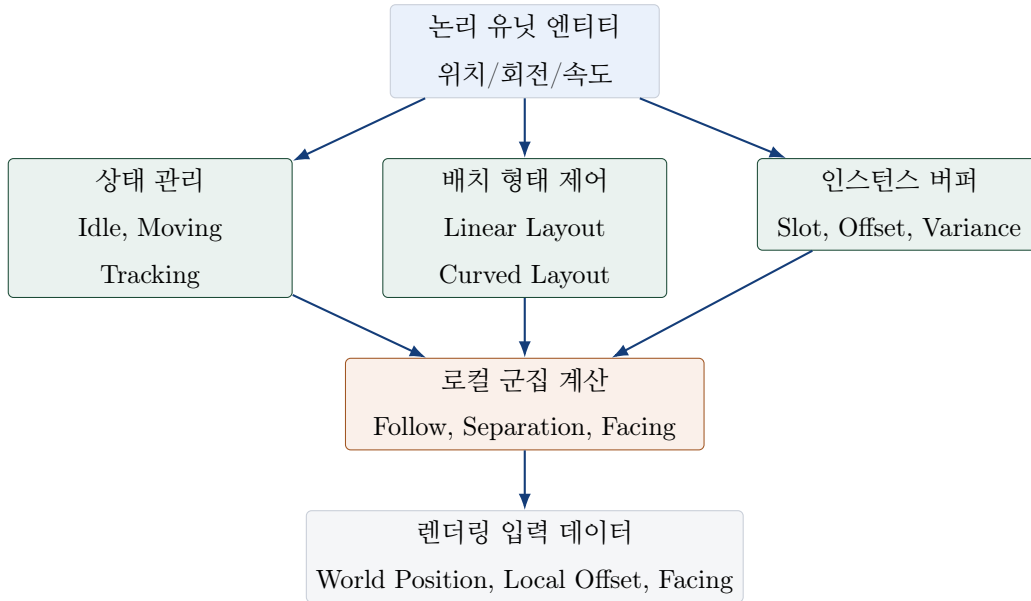


그림 4.3: 군집형 인스턴스 시뮬레이션 구조

4.4 렌더링 파이프라인 설계

렌더링 파이프라인은 CPU가 정적 인스턴스 데이터 또는 ECS 기반 데이터 갱신 결과에서 필요한 인스턴스 데이터를 수집하고, GPU가 컬링 및 행렬 생성을 담당하도록 설계한다. GPU 기반 컬링 단계는 각 인스턴스의 월드 위치를 계산하고 카메라 프러스텀 밖의 인스턴스를 제거한다. 선별된 인스턴스 수는 Indirect Arguments Buffer 또는 이에 준하는 제출 데이터에 기록되며, 최종 렌더링은 BRG 기반 Draw Command 또는 Indirect Draw 제출 경로로 수행된다.

본 설계에서 렌더링 계층은 단순한 화면 출력 기능이 아니라 실험의 핵심 대상이다. 실험에서는 Render-only BRG 조건을 통해 순수 렌더링 제출 비용을 확인하고, ECS-fed BRG 조건을 통해 ECS 기반 데이터 갱신이 결합되었을 때의 추가 비용을 분석한다. 또한 인스턴스 유형별 분할 제출 방식과 통합 Draw List 제출 방식을 비교하여, 인스턴스 유형 수 증가가 Render Thread, GPU Time, Draw Command Count에 미치는 영향을 분석한다. Shadow ON을 기본 조건으로 설정하고, 대표 조건에서 Shadow OFF를 추가 측정하여 ShadowCaster Pass의 영향을 분리한다.

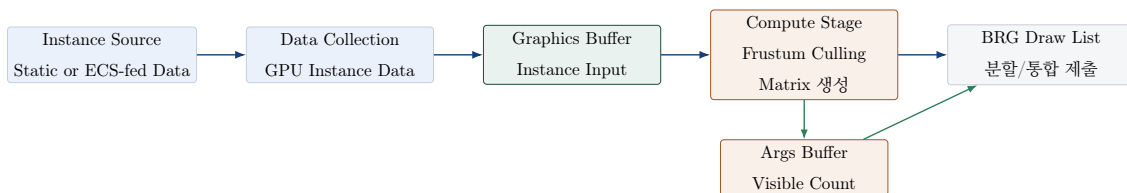


그림 4.4: BRG 기반 렌더링 제출 파이프라인

4.5 실험 조건 구성

실험 설계는 같은 3D 인스턴스 장면을 서로 다른 BRG 기반 제출 조건으로 실행하고, 프레임타임과 렌더링 제출 비용의 변화를 비교하는 방식으로 구성한다. 비교의 핵심은 단순 FPS 차이가 아니라, 렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, 제출 방식 변화에 따라 병목이 Main Thread, Render Thread, GPU 단계 중 어디로 이동하는지를 정량적으로 확인하는 것이다.

개별 GameObject 또는 per-object draw 방식은 교수자 제공 OpenGL 예비 벤치마크를 통해 구조적 한계가 확인된 선행 검토 대상으로 두고, 본 프로젝트에서는 직접 구현 범위에서 제외한다. 이에 따라 실험 조건은 BRG 기반 렌더링 제출 구조 내부 비교에 집중한다.

표 4.1: BRG 기반 실험 조건 비교 설계

구분	데이터 갱신 방식	렌더링 제출 방식	비교 목적
E0	선행 벤치마크 인용	per-object draw, instancing, indirect 비교	개별 draw 방식 직접 구현 제외의 근거 제시
E1	정적 또는 단순 인스턴스 데이터	Render-only BRG	순수 BRG 렌더링 제출 비용 확인
E2	ECS 기반 데이터 갱신	ECS-fed BRG	논리 유닛 수 증가가 Main Thread 및 제출 구조에 미치는 영향 확인
E3	ECS 기반 데이터 갱신	인스턴스 유형별 분할 BRG 제출	인스턴스 유형 수 증가에 따른 Draw Command 및 Render Thread 비용 확인
E4	ECS 기반 데이터 갱신	통합 Draw List 기반 BRG 제출	통합 제출 구조의 병목 완화 가능성 확인
E5	대표 조건 반복 측정	Shadow OFF 보조 조건	ShadowCaster Pass에 따른 GPU 및 제출 비용 분리

제5장 구현 계획

5.1 현재 프로젝트 기반

본 프로젝트는 Unity 6000.3.9f1 기반이며, `unity.entities`, `unity.feature.ecs`, URP 17.3.0 패키지를 사용한다. 본 프로젝트에서는 기존 프로젝트 환경을 바탕으로 대규모 3D 인스턴스 시뮬레이션 및 BRG 중심 렌더링 제출 실험을 수행할 수 있는 구조를 단계적으로 정리한다.

개별 GameObject 또는 per-object draw 방식은 인스턴스 수 증가에 따라 draw call과 CPU 제출 비용이 선형적으로 증가하는 구조적 한계를 가진다. 해당 방식은 교수자 제공 OpenGL 예비 벤치마크와 기존 렌더링 파이프라인의 특성을 통해 기본 한계가 확인되었으므로, 본 프로젝트에서는 이를 직접 구현하는 데 범위를 소모하지 않는다. 대신 실제 구현 범위는 BRG 기반 렌더링 제출 구조 내부의 조건별 비교에 집중한다.

주요 구현 영역은 다음과 같이 계획한다.

표 5.1: 주요 구현 영역과 설계상 역할

구현 영역	설계상 역할
논리 유닛 데이터 모델	논리 유닛의 상태, 이동 목표, 군집 파라미터, 인스턴스 버퍼, Snapshot 데이터 구조 정의
초기 인스턴스 배치 계층	인스턴스 슬롯, 랜덤 편차, 초기 로컬 위치를 생성하여 반복 가능한 실험 조건 구성
논리 유닛 상태 갱신 계층	논리 유닛의 이동, 회전, 목표 추적, 배치 형태 전환 상태 계산
내부 인스턴스 갱신 계층	내부 인스턴스의 로컬 이동, 간격 유지, 분리 힘, 이동 지연, 방향 보정 처리
공간 처리 계층	공간 그리드 기반 공간 질의, 관심 영역 선정, 장애물 페널티, 우선순위 계산 담당
렌더링 데이터 전달 계층	정적 인스턴스 데이터 또는 ECS 기반 데이터 갱신 결과를 GPU 버퍼와 BRG 제출 경로에 전달
GPU 킬링 및 행렬 생성 계층	프러스텀 킬링, 인스턴스 행렬 생성, visible instance count 기록 담당
BRG 제출 구조 비교 계층	Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건 구성
Shadow 조건 비교 계층	Shadow ON 기본 조건과 Shadow OFF 보조 조건을 구성하여 ShadowCaster Pass에 따른 추가 부하 분리

5.2 구현 단계

표 5.2: 단계별 구현 계획

단계	목표	세부 작업	산출물
1단계	기준선 정리	현재 씬, 패키지, 기존 군집형 인스턴스 구조를 정리하고 렌더링 인스턴스 수, 해상도, Shadow 설정, 카메라 조건 고정. per-object draw 방식은 선행 벤치마크 근거로 직접 구현 대상에서 제외	기준 씬, 설정표
2단계	ECS 기반 데이터 갱신 안정화	논리 유닛 상태, 이동 명령, 배치 형태 전환, 공간 처리 시스템을 텍 단위로 안정화	데이터 갱신 데모
3단계	Render-only BRG 조건 구성	시뮬레이션 부하를 최소화한 상태에서 BRG 기반 대량 인스턴스 제출 경로를 구성하여 순수 렌더링 제출 비용을 측정할 수 있게 구현	Render-only BRG 실험 조건
4단계	ECS-fed BRG 조건 구성	ECS 기반 데이터 갱신 결과를 BRG 제출 경로에 연결하여 논리 유닛 수 증가가 Main Thread와 렌더 제출 구조에 미치는 영향 확인	ECS-fed BRG 실험 조건
5단계	제출 방식 비교 조건 구성	인스턴스 유형별 분할 제출 조건과 통합 Draw List 기반 제출 조건을 구성하여 Draw Command 증가와 Render Thread 병목 비교	분할 제출/통합 제출 비교 조건
6단계	Shadow 조건 구성	Shadow ON을 기본 조건으로 유지하고, 대표 인스턴스 수에서 Shadow OFF 조건을 추가하여 ShadowCaster Pass의 영향을 분리	Shadow ON/OFF 비교 조건

단계	목표	세부 작업	산출물
7단계	성능 측정 자동화	렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, visible instance count, frame time, Draw Command, GPU time을 로그로 수집	CSV 로그, 그래프
8단계	결과 정리 및 발표 준비	실험 결과와 병목 분석을 정리하고, 최종보고서 및 발표 자료에 반영할 수 있도록 표, 그래프, Profiler 캡처 자료 구성	보고서, 발표 자료

5.3 설계상 핵심 결정

- 본 프로젝트의 구현 범위는 개별 GameObject 또는 per-object draw 방식의 재현이 아니라, BRG 기반 렌더링 제출 구조의 조건별 차이를 분석하는 데 둔다.
- 모든 렌더링 인스턴스를 독립 엔티티로 관리하기보다, 논리 유닛 엔티티와 내부 버퍼 구조를 조합하여 군집형 인스턴스 단위 상태 관리 비용을 낮춘다.
- Render-only BRG 조건과 ECS-fed BRG 조건을 분리하여, 순수 렌더링 제출 비용과 ECS 기반 데이터 갱신 비용을 구분해 측정한다.
- 인스턴스 유형별 분할 제출 조건과 통합 Draw List 제출 조건을 비교하여, Draw Command 구성 방식이 Render Thread와 GPU Time에 미치는 영향을 분석한다.
- Shadow ON을 기본 조건으로 설정하되, 대표 조건에서 Shadow OFF를 보조 측정하여 ShadowCaster Pass에 따른 부하를 분리한다.
- 렌더링은 CPU에서 모든 행렬을 직접 생성하지 않고 GPU 기반 켤링과 행렬 생성 단계를 활용하며, BRG 및 GPU Indirect Rendering 기반 제출 경로를 통해 Draw Command 증가를 통제한다.
- 실험 결과는 단일 FPS가 아니라 frame time, Main Thread, Render Thread, GPU Time, Draw Command, visible instance count를 함께 기록하여 설계 선택의 타당성을 설명한다.

5.4 기능 명세

표 5.3: 주요 기능 명세

기능	입력	처리	우선순위
논리 유닛 생성	인스턴스 수, 열 수, 간격, 시드	슬롯과 인스턴스 편차를 생성하고 버퍼에 저장	높음
이동 명령	목표 위치	논리 유닛 회전, 이동 속도, 도착 판정 처리	높음
배치 형태 전환	각속도, 상태 정보	직선 이동형 및 회전 이동형 배치 상태를 전환하고 부드럽게 보간	중간
공간 처리	그룹 정보, 탐색 범위, 장애물	공간 그리드에서 관심 영역과 장애물 페널티를 계산	중간
Render-only BRG 렌더링	정적 또는 단순 인스턴스 데이터	시뮬레이션 부하를 최소화한 상태에서 BRG 기반 대량 렌더링을 수행	높음
ECS-fed BRG 렌더링	ECS 기반 인스턴스 위치, 방향, 상태	데이터 갱신 결과를 GPU 버퍼와 BRG 제출 경로에 연결	높음
제출 방식 비교	인스턴스 유형 수, Draw List 구성 방식	인스턴스 유형별 분할 제출과 통합 Draw List 제출의 성능 차이를 측정	높음
Shadow 조건 비교	Shadow ON/OFF 설정	ShadowCaster Pass 포함 여부에 따른 GPU 및 제출 비용 차이를 측정	중간
성능 측정	실험 조건	FPS, frame time, Draw Command, visible instance count, GPU time 기록	높음

제6장 실험 및 평가 계획

6.1 실험 범위 설정

본 프로젝트의 실험은 대규모 3D 인스턴스 장면에서 BRG 기반 렌더링 제출 구조가 어떤 조건에서 효율적으로 동작하고, 병목이 어느 단계로 이동하는지를 확인하는 데 초점을 둔다. 개별 GameObject 또는 per-object draw 방식은 인스턴스 수 증가에 따라 draw call과 CPU 제출 비용이 선형적으로 증가하는 구조적 한계를 가진다. 해당 방식은 교수자 제공 OpenGL 예비 벤치마크와 기존 렌더링 파이프라인 특성을 통해 기본 한계가 확인되었으므로, 본 프로젝트에서는 이를 직접 구현 범위에 포함하지 않는다.

따라서 실험의 중심은 기존 방식과의 단순 비교가 아니라, BRG 기반 렌더링 제출 구조 내부에서 조건을 달리했을 때 Main Thread, Render Thread, GPU Time, Draw Command, visible instance count가 어떻게 변화하는지 분석하는 것이다. 이를 위해 Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건을 구성한다.

6.2 실험 변인

실험 변인은 총 렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, BRG 제출 방식, Shadow 조건으로 구분한다. 여기서 인스턴스 유형은 모델, 머티리얼, 모션, 논리구조가 서로 다른 3D 인스턴스 그룹을 의미한다. 이 중 Mesh, Material, Shader, LOD 구성처럼 렌더링 제출 단위에 직접 영향을 주는 요소는 렌더 타입으로 구분할 수 있으나, 본 착수보고서에서는 보다 넓은 의미의 인스턴스 유형을 기본 용어로 사용한다.

표 6.1: 실험 독립 변인

변수	정의	실험 수준 및 의미
N_{render}	총 렌더링 인스턴스 수	화면에 표시될 수 있는 전체 3D 인스턴스 수. 초기 실험은 1,000, 5,000, 10,000, 20,000 단위로 구성하고, 장비 여유가 있으면 50,000 이상까지 확장
N_{unit}	논리 유닛 수	ECS 기반 데이터 갱신 계층에서 관리되는 논리 유닛의 수. Render-only BRG 조건과 ECS-fed BRG 조건을 비교할 때 Main Thread 부하를 확인하는 기준으로 사용
$K_{instance}$	인스턴스 유형 수	모델, 머티리얼, 모션, 논리구조가 다른 인스턴스 그룹의 수. 인스턴스 유형별 분할 제출과 통합 Draw List 제출을 비교하는 기준으로 사용
R_{submit}	BRG 제출 방식	Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출로 구분
S_{shadow}	Shadow 조건	Shadow ON을 기본 조건으로 두고, 대표 조건에서 Shadow OFF를 추가 측정하여 ShadowCaster Pass의 영향을 분리

6.3 측정 지표

본 프로젝트의 평가는 단순히 평균 FPS만 확인하는 방식이 아니라, 병목이 어느 단계에서 발생하는지를 분리하여 관찰하는 방식으로 수행한다. 특히 Shadow ON을 기본 조건으로 설정하여 실제 렌더링 부하가 포함된 상태에서 측정하고, 일부 대표 조건에서는 Shadow OFF 측정을 추가하여 순수 렌더 제출 비용과 ShadowCaster Pass의 영향을 구분한다.

표 6.2: 성능 평가 지표

지표	의미	수집 방법
평균 FPS	전체 실행 성능을 직관적으로 확인하기 위한 지표	Unity Stats 또는 자체 로그
Frame Time	프레임 안정성과 순간적인 프레임 톱 현상 확인	Unity Profiler, Recorder API
Main Thread Time	ECS 기반 데이터 갱신, 논리 유닛 처리, 인스턴스 데이터 수집 비용 확인	Unity Profiler
Render Thread Time	BRG 기반 Draw Command 구성 및 렌더 제출 비용 확인	Unity Profiler
GPU Time	Main Pass 및 ShadowCaster Pass를 포함한 실제 GPU 처리 병목 확인	Unity Profiler GPU 모듈 또는 외부 분석 도구
Draw Command Count	제출 방식에 따른 명령 수 증가 여부 확인	Unity Stats, Frame Debugger, 로그
Visible Instance Count	컬링 후 실제 렌더링 대상이 되는 인스턴스 수 확인	Args Buffer 또는 자체 로그

6.4 실험 조건

실험 조건은 BRG 기반 렌더링 제출 구조 내부 비교를 중심으로 구성한다. per-object draw 방식은 직접 구현하지 않고, 교수자 제공 OpenGL 예비 벤치마크를 선행 검토 자료로 활용한다. 실제 구현 실험은 Render-only BRG 조건과 ECS-fed BRG 조건을 분리하여 순수 렌더 제출 비용과 ECS 기반 데이터 갱신 비용을 구분하고, 인스턴스 유형별 분할 제출과 통합 Draw List 제출을 비교하여 Draw Command 구성 방식에 따른 병목 차이를 확인한다.

표 6.3: BRG 기반 실험 조건

조건	데이터 갱신 방식	렌더링 제출 방식	비교 목적
E0	선행 벤치마크 인용	per-object draw, instancing, indirect 비교	개별 draw 방식 직접 구현 제외의 근거 제시
E1	정적 또는 단순 인스턴스 데이터	Render-only BRG	시뮬레이션 부하가 최소화된 상태에서 순수 BRG 렌더링 제출 비용 확인
E2	ECS 기반 데이터 갱신	ECS-fed BRG	논리 유닛 수 증가가 Main Thread와 BRG 제출 구조에 미치는 영향 확인
E3	ECS 기반 데이터 갱신	인스턴스 유형별 분할 BRG 제출	인스턴스 유형 수 증가에 따른 Draw Command 및 Render Thread 비용 확인
E4	ECS 기반 데이터 갱신	통합 Draw List 기반 BRG 제출	통합 제출 구조가 분할 제출 대비 병목을 완화할 수 있는지 확인
E5	대표 조건 반복 측정	Shadow OFF 보조 조건	Shadow ON 기본 조건과 비교하여 ShadowCaster Pass에 따른 GPU 및 제출 비용 분리

6.5 실험 절차

각 실험은 동일한 씬, 동일한 해상도, 동일한 카메라 조건, 동일한 난수 시드를 기준으로 수행한다. 성능 측정은 일시적인 로딩 또는 초기화 비용이 포함되지 않도록 워밍업 구간을 둔 뒤 진행한다.

1. 렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, 제출 방식, Shadow 조건을 실험 설정값으로 고정한다.
2. 동일한 카메라 위치와 이동 경로를 사용하여 visible instance count 변화가 실험 간 비교

가능하도록 한다.

3. 각 실험은 워밍업 10초 후 60초 동안 측정한다.
4. 동일 실험을 3회 반복하고 평균, 최소, 최대, 95퍼센타일 및 99퍼센타일 frame time을 기록한다.
5. Shadow ON을 기본 조건으로 측정하되, 대표 인스턴스 수에서는 Shadow OFF 조건을 추가 측정한다.
6. 병목이 확인되면 Unity Profiler 캡처와 Frame Debugger 결과를 보관하여 결과 해석에 활용한다.

6.6 기대 결과와 해석 방법

Render-only BRG 조건에서는 시뮬레이션 부하가 최소화된 상태에서 BRG 기반 렌더링 제출 구조의 기본 비용을 확인할 수 있을 것으로 예상된다. ECS-fed BRG 조건에서는 논리 유닛 수 증가에 따라 Main Thread의 데이터 갱신 비용이 증가할 수 있으며, 이 차이를 Render-only BRG 조건과 비교함으로써 시뮬레이션 데이터 갱신 비용을 분리할 수 있다.

인스턴스 유형별 분할 제출 조건에서는 유형 수가 증가할수록 Draw Command 구성과 Render Thread 제출 비용이 증가할 가능성이 있다. 반면 통합 Draw List 제출 조건에서는 제출 구조가 단순화되어 Render Thread 병목이 완화될 수 있으나, 단일 Draw List 내부의 인덱싱과 데이터 관리 비용이 증가할 수 있다. 따라서 최종 해석은 단순히 FPS가 높은 조건을 찾는 것이 아니라, 어떤 조건에서 Main Thread, Render Thread, GPU 중 어느 단계가 병목이 되는지를 중심으로 작성한다.

6.7 결과 기록 방식

실험 결과는 실제 측정값을 표와 그래프로 정리한다. 결과 기록은 별도의 CSV 로그 또는 분석 스프레드시트로 관리하며, 각 행은 하나의 실험 실행을 의미한다. 주요 열은 실험 조건 (E1-E5), 총 렌더링 인스턴스 수(N_{render}), 논리 유닛 수(N_{unit}), 인스턴스 유형 수($K_{instance}$), BRG 제출 방식(R_{submit}), Shadow 조건(S_{shadow}), 반복 회차, 평균 FPS, 평균 frame time, 95퍼센타일 및 99퍼센타일 frame time, Main Thread time, Render Thread time, GPU time, Draw Command Count, visible instance count, 해석 메모로 구성한다. 이를 통해 단순 평균 성능뿐 아니라 프레임 안정성, 렌더링 제출 비용, GPU 처리 비용, ShadowCaster Pass 영향까지 함께 분석할 수 있도록 한다.

6.8 보고서 반영 방식

중간보고서에는 구현 완료 범위와 초기 측정 가능 항목을 포함하고, 최종보고서에는 BRG 기반 실험 조건별 비교 그래프를 중심으로 병목 이동과 제출 방식별 성능 차이를 정리한다. 성능 그래프는 총 렌더링 인스턴스 수(N_{render})를 x축, frame time 또는 FPS를 y축으로 구성하며, Draw Command와 visible instance count는 보조 표로 제시한다.

또한 논리 유닛 수(N_{unit}), 인스턴스 유형 수($K_{instance}$), 제출 방식(R_{submit}), Shadow 조건(S_{shadow}) 변화에 따른 Main Thread, Render Thread, GPU Time의 차이를 함께 제시한다. 이를 통해 BRG 중심 렌더링 최적화 구조의 적용 범위와 한계를 설명한다.

제7장 추진 체계 및 역할 분담

7.1 추진 일정

본 프로젝트는 학과에서 제시한 공식 제출 일정을 기준으로 진행하되, 구현과 실험 분석은 지도교수 지도하에 추진하는 학회 논문 제출 일정을 함께 고려하여 공식 제출일보다 앞서 완료하는 것을 목표로 한다. 공식 일정은 착수보고서 및 지도확인서 제출, 중간보고서 및 중간평가표 제출, 최종보고서 및 최종평가표 제출, 졸업과제 발표심사, 결과물 업로드 순서로 구성된다. 자체 일정은 BRG 중심 렌더링 제출 구조 구현, ECS 기반 인스턴스 데이터 갱신 안정화, 성능 데이터 수집 및 분석, 논문형 보고서 작성 순으로 진행한다.

표 7.1: 공식 일정 및 자체 추진 일정

기간	목표	세부 내용
2026.04.28-05.15	착수보고서 및 지도확인서 제출	주제 확정, 연구 범위 정리, 요구사항 분석, 시스템 설계, 실험 방향 설정, 지도교수 검토 후 착수보고서 및 지도확인서 제출
2026.05.16-06.02	기반 구조 정리	Unity 프로젝트 환경 고정, 렌더링 인스턴스 수, 해상도, Shadow 설정, 카메라 조건 등 반복 실험을 위한 공통 조건 정리
2026.06.03-06.23	ECS 기반 데이터 갱신 안정화	논리 유닛 상태 갱신, 내부 인스턴스 버퍼 구조, 배치 형태 전환, 공간 처리 계층을 안정화, BRG 실험에 전달할 데이터 흐름 정리

기간	목표	세부 내용
2026.06.24-07.10	BRG 렌더링 제출 조건 구현	Render-only BRG, ECS-fed BRG, 인스턴스 유형별 분할 제출, 통합 Draw List 제출 조건 구성, 기본 렌더링 동작 검증
2026.07.11-07.17	중간보고서 및 중간평가표 제출	중간보고서 제출을 위해 구현 진행 상황, 실험 조건, 초기 측정 가능 항목, 남은 구현 범위 정리, 중간평가표 제출
2026.07.18-07.19	전체 구현 완료	성능 측정에 필요한 모든 실험 조건을 실행 가능한 상태로 정리하고, 실험군 간 입력 조건과 로그 형식을 통일
2026.07.20-08.09	데이터 수집 및 분석 완료	렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, 제출 방식, Shadow 조건별 성능 데이터 수집, frame time, Draw Command, GPU time, visible instance count 분석
2026.08.10-08.16	논문 작성	실험 결과 표와 그래프 정리, 병목 이동 분석, BRG 제출 구조 비교, 한계 및 개선 방향을 포함한 논문형 보고서 작성
2026.08.17-09.18	최종보고서 및 최종평가표 제출	최종 실험 결과, 분석 그래프, 구현 설명, 결론을 보완하여 최종보고서와 최종평가표 제출
2026.09.19-09.29	발표심사 준비	발표 자료 제작, 데모 안정화, 실험 결과 요약, 질의응답 예상 문항 정리, 발표 리허설 수행
2026.09.30 또는 10.01	졸업과제 발표심사	테크위크 발표심사에서 구현 결과, 실험 설계, 성능 분석 결과, 향후 발전 방향 발표

기간	목표	세부 내용
2026.10 중순까지	결과물 업로드	소스 코드, 실행 파일, 최종보고서, 발표 자료, 실험 로그, 그래프 자료를 정리하여 결과물 업로드

7.2 역할 분담

본 프로젝트는 BRG 기반 렌더링 최적화 실험을 중심으로 진행되며, 이를 위해 ECS 기반 인스턴스 데이터 파이프라인, 군집형 인스턴스 제어, 휴머노이드 모션 제어, 성능 측정 및 실험 조건 구성이 함께 필요하다. 역할 분담은 현재까지의 구현 경험과 기술적 기여를 바탕으로 하되, 향후 구현 완료, 실험 데이터 수집, 분석 단계까지 이어질 수 있도록 구성한다. 보고서 작성 및 문서 검수 업무는 본 역할 분담에서 제외하고, 기술 구현과 실험 수행 역할을 기준으로 정리한다.

표 7.2: 팀원 역할 분담

팀원	주 역할	담당 내용
202255658 김재식	ECS 인프라 및 핵심 BRG 렌더링 아키텍처	ECS 기반 데이터 파이프라인 설계, 논리 유닛과 인스턴스 버퍼 구조 정의, 시뮬레이션 데이터를 GPU 렌더링 계층으로 전달하기 위한 데이터 마샬링 구조 구축, Render-only BRG와 ECS-fed BRG의 기반 구조 설계, 전체적인 BRG Draw Command 제출 로직 관리, 병목 분석에 필요한 핵심 렌더링 아키텍처 정리
202155593 이천서	시뮬레이션 로직 고도화 및 실험 자동화 시스템	ECS 기반 군집형 인스턴스 제어 구조에서 휴머노이드 모션 데이터가 안정적으로 적용되도록 관리, 애니메이션 위상과 상태 전환 로직이 다수의 인스턴스 유형에서도 일관되게 동작하도록 검토, 실험 시나리오별 FPS, frame time, GPU time 등의 성능 지표를 자동 수집하는 벤치마크 도구와 CSV 및 그래프 산출 구조 구성

팀원	주 역할	담당 내용
202255568 안지홍	인스턴스 다양성 개발 및 유형별 BRG 최적화	모델, 머티리얼, 모션 상태가 서로 다른 다종의 3D 인스턴스 유형을 구성하고 실험 조건에 맞게 최적화, 인스턴스 유형 수 증가에 따른 분할 제출(E3)과 통합 Draw List 제출(E4)의 성능 차이를 검증할 수 있도록 유형별 BRG 제출 구조와 전용 Draw List 관리 방식 구현
공통	학술적 검증 및 결과 정리	각 구현 영역을 통합하여 동일한 실험 조건에서 렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, Shadow 조건, BRG 제출 방식별 성능 측정이 가능하도록 구현, 수집된 정량적 지표를 바탕으로 Main Thread, Render Thread, GPU 단계의 병목 이동 현상을 분석, 최종보고서 및 발표 자료를 공동 검토

7.3 협업 방식

코드는 Git 저장소에서 관리하고, 실험 조건과 결과는 CSV 또는 스프레드시트 형태로 저장한다. 보고서는 LaTeX 소스로 관리하여 착수, 중간, 최종보고서가 같은 양식과 용어를 공유하도록 한다. 주요 의사결정은 이슈 또는 문서에 남겨 중간보고서와 최종보고서 작성 시 근거로 재사용한다. 특히 렌더링 인스턴스 수, 논리 유닛 수, 인스턴스 유형 수, Shadow 설정, 제출 방식과 같은 실험 조건은 재현 가능성을 위해 별도 기록으로 관리한다.

제8장 기대 효과 및 향후 발전 방향

8.1 기대 효과

본 프로젝트의 기대 효과는 기술적 효과와 교육적 효과로 나눌 수 있다. 기술적으로는 Unity 기반 대규모 3D 인스턴스 장면에서 BRG와 GPU Indirect Rendering 중심의 렌더링 제출 구조를 검증하고, ECS 기반 데이터 갱신 계층과 결합했을 때의 병목 이동을 정량적으로 분석할 수 있다. 교육적으로는 컴퓨터 구조, 병렬 처리, 그래픽스 파이프라인, 실험 설계를 실제 시스템 구현에 연결하는 경험을 제공한다.

- 대규모 3D 인스턴스 처리에서 객체 지향 방식과 데이터 지향 방식의 차이를 정량적으로 설명할 수 있다.
- CPU 시뮬레이션 병목, Render Thread 제출 병목, GPU 처리 병목을 분리하여 분석하는 실험 방법을 확보한다.
- BRG 기반 Draw Command 구성, 인스턴스 유형 다양성, Shadow ON 조건이 성능에 미치는 영향을 실험적으로 분석할 수 있다.
- 향후 군중 시각화, 대규모 공간 시뮬레이션, 디지털 트윈, 전략적 시각화 프로젝트로 확장 가능한 렌더링 및 데이터 갱신 기반을 마련한다.
- 최종 결과를 단순 시연 영상이 아니라 성능 지표와 설계 근거가 있는 산출물로 정리할 수 있다.

8.2 향후 발전 방향

최종보고서 이후에는 다음 방향으로 확장할 수 있다.

1. **LOD 및 애니메이션 확장:** 거리 기반 LOD, GPU 애니메이션, 간소화된 그림자 전략을 적용한다.
2. **LOD 레벨별 BRG 그룹 관리 방식 비교:** 복수 LOD 레벨을 하나의 BRG 그룹 내 버퍼에 통합하여 인덱스로 관리하는 방식과, 각 LOD 레벨을 별도의 BRG 그룹으로 분리하여

관리하는 방식을 비교한다. 전자는 그룹 수를 줄여 제출 구조를 단순화할 수 있지만 단일 그룹 내부의 인덱싱과 데이터 관리가 복잡해질 수 있다. 후자는 LOD 레벨별 관리가 직관적이지만 그룹 증가에 따른 제출 및 관리 오버헤드가 발생할 수 있다. 따라서 LOD 전환 시 그룹 간 데이터 이동 비용과 단일 그룹 내 복잡한 인덱싱 연산 비용을 비교하는 것은 실무적인 렌더링 최적화 연구로 확장될 수 있다.

3. **렌더링 제출 구조 고도화:** BRG 기반 Draw List 구성 방식을 확장하여 인스턴스 유형별 분할 제출, 통합 제출, 컬링 결과 반영 방식의 차이를 추가 분석한다.
4. **시뮬레이션 데이터 동기화:** 현재 텍 기반 Snapshot 구조를 활용해 결정론적 상태 갱신과 재현 가능한 실험 로그 관리 가능성을 검토한다.

참고 문헌 및 자료

- [1] Unity Technologies, “Unity Entity Component System,” <https://unity.com/ecs>.
- [2] Unity Technologies, “DOTS concepts, features, samples and resources for Unity 6,” <https://unity.com/resources/dots-concepts-features-samples-resources-unity-6>.
- [3] Unity Technologies, “Entities package documentation,” <https://docs.unity3d.com/Packages/com.unity.entities@latest>.
- [4] Unity Technologies, “Graphics.RenderMeshIndirect scripting API,” <https://docs.unity3d.com/ScriptReference/Graphics.RenderMeshIndirect.html>.